

Documentacao do Projeto

Simulador de Escalonamento de Tarefas — Sistemas Operacionais

1. Visao geral

O simulador reproduz o escalonamento de um conjunto de tarefas em um unico processador, sob tempo discreto. O modelo implementado e o de escalonamento de curto prazo: um conjunto de tarefas ingressa ao longo do tempo, disputa o processador segundo uma politica, e o simulador registra o instante de cada despacho ate que todas concluem, alem dos intervalos ociosos, das trocas de contexto e dos periodos em que cada tarefa fica bloqueada no recurso.

A partir desse registro o programa apresenta, por tarefa e em media, o tempo de execucao, o tempo de processamento, o tempo de espera e o tempo ate a primeira execucao, alem do diagrama de tempo.

Sobre esse nucleo o simulador trata ainda o recurso de uso exclusivo, o que permite reproduzir a inversao de prioridades e comparar os dois protocolos de correcao, heranca e teto, sobre o mesmo conjunto de tarefas.

2. Separacao entre politica e mecanismo

Este e o eixo do projeto.

O **mecanismo** e o laco de simulacao, em `simulador/motor.py`. Ele avanca o relógio, monta o conjunto de tarefas prontas, cobra a troca de contexto, controla a posse do recurso, suspende e libera tarefas e recalcula a prioridade efetiva sob envelhecimento, heranca e teto. O mecanismo e unico: nao existe um laco por algoritmo.

A **politica** e a decisao de qual tarefa recebe o processador, em `simulador/politicas.py`. Uma politica e uma funcao que recebe a lista de tarefas prontas e devolve a tarefa escolhida. Ela nao conhece o relógio, o custo da troca de contexto, o recurso nem os protocolos de correcao.

A fronteira entre as duas e a assinatura da funcao de escolha:

```
def escolher(prontas: list[Tarefa], fila: list[Tarefa]) -> Tarefa
```

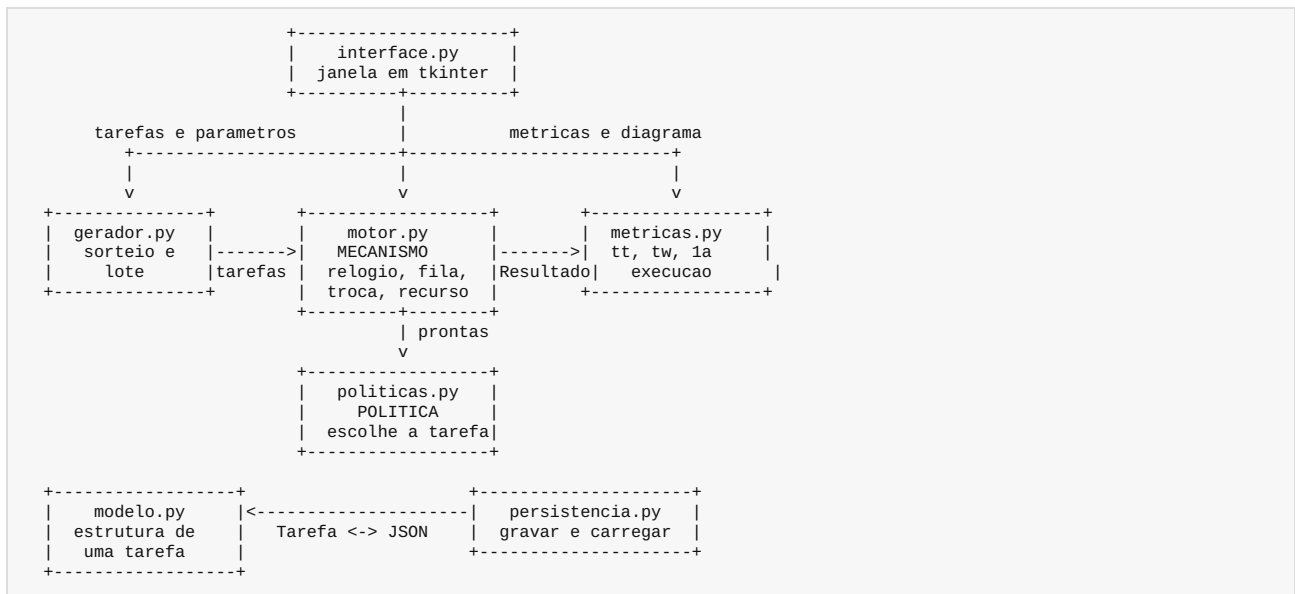
Cada uma das seis politicas cabe em uma linha, porque a unica coisa que muda de um algoritmo para outro e o criterio de ordenacao sobre `prontas`:

Algoritmo	Criterio
FCFS	menor ingresso
SJF	menor tp
SRTF	menor restante
RR	primeiro da fila circular mantida pelo motor
PRIOc, PRIOp	maior prioridade_efetiva

O que distingue um algoritmo cooperativo de um preemptivo nao esta na politica, e sim em duas propriedades declaradas na estrutura `Algoritmo`: `preemptivo` e `usa_quantum`. O motor as consulta para decidir quando reavaliar a escolha — a cada unidade de tempo nos preemptivos sem quantum, ao fim da fatia sob Round-Robin, e apenas quando o processador se libera nos cooperativos.

O motor nao conhece o criterio de escolha, e a politica nao conhece o relógio.

3. Diagrama de modulos



O fluxo de dados percorre a interface, que reúne as tarefas e os parametros; o motor, que executa a simulacao consultando a politica a cada decisao; e o modulo de metricas, que converte o resultado nos numeros apresentados. O modulo `modelo.py` fornece a estrutura de dados comum a todos, e `persistencia.py` converte essa estrutura de e para JSON.

4. Estrutura de uma tarefa

Definida em `simulador/modelo.py`. Os campos se dividem em dados de definicao, informados por quem executa o simulador, e campos de estado, mantidos pelo motor durante a simulacao.

Campos de definicao

Campo	Tipo	Funcao
<code>id</code>	inteiro	Identificador da tarefa
<code>ingresso</code>	inteiro	Instante em que a tarefa surge
<code>tp</code>	inteiro	Tempo de processamento demandado
<code>prioridade</code>	inteiro	Prioridade base; maior valor, prioridade mais alta
<code>uso_de_r</code>	tupla ou <code>None</code>	Quando a tarefa obtém R e por quanto tempo o mantém

Campos de estado

Campo	Tipo	Funcao
<code>executado</code>	inteiro	Unidades uteis ja processadas
<code>conclusao</code>	inteiro ou <code>None</code>	Instante em que a tarefa terminou
<code>primeira_exec</code>	inteiro ou <code>None</code>	Instante do primeiro despacho efetivo
<code>suspensa</code>	booleano	Verdadeiro enquanto a tarefa espera o recurso R
<code>detem_r</code>	booleano	Verdadeiro enquanto a tarefa mantém o recurso R
<code>liberou_r</code>	booleano	Verdadeiro depois da devolucao; impede novo pedido
<code>ultimo_despacho</code>	inteiro ou <code>None</code>	Referencia do envelhecimento
<code>prioridade_efetiva</code>	inteiro	Prioridade apos envelhecimento, heranca ou teto

As faixas validas sao verificadas pela funcao `validar`: ingresso nao negativo, duracao positiva e secao critica contida

na duracao da tarefa.

5. Interface dos modulos

modelo.py

Elemento	Recebe	Devolve	O que faz
Tarefa	os campos de definicao	—	Estrutura de dados de uma tarefa
Tarefa.reiniciar	—	—	Devolve a tarefa ao estado anterior a qualquer simulacao
Tarefa.copia_limpa	—	Tarefa	Nova tarefa com os mesmos dados e estado zerado
validar	Tarefa	—	Levanta ValueError com mensagem clara se algum valor estiver fora da faixa

politicas.py

Elemento	Recebe	Devolve	O que faz
fcfs, sjf, srtf, round_robin, prioridade	prontas, fila	Tarefa	Aplicam o criterio de escolha do algoritmo
Algoritmo	sigla, nome, funcao, propriedades	—	Descreve uma politica e o que o motor precisa saber sobre ela
ALGORITMOS	—	dicionario	Mapeia a sigla ao algoritmo correspondente

motor.py

Elemento	Recebe	Devolve	O que faz
Motor	tarefas, sigla, quantum, custo, protocolo, alfa	—	Prepara a simulacao e recusa combinacoes invalidas
Motor.simular	—	Resultado	Executa o laco ate que todas as tarefas concluem
Motor._despachar	instante, tarefa, orcamento	instante	Executa a tarefa ate que ela perca o processador
Motor._atualizar_prioridades	instante, prontas	—	Recalcula a prioridade efetiva sob os tres mecanismos
Motor._atualizar_fila	instante, execucao	—	Mantem a ordem da fila circular do Round-Robin
Motor._registrar_suspensas	instante	—	Guarda quais tarefas estao bloqueadas no recurso a cada instante
Resultado.eficiencia	—	float ou None	Calcula tq/(tq+ttc), ou None fora do Round-Robin

metricas.py

Elemento	Recebe	Devolve	O que faz
calcular	Resultado	Metricas	Calcula as quatro grandezas por tarefa e em media
sequencia_de_execucao	Resultado	texto	Reduz a linha do tempo ao formato t1[0,5) t4[5,8)

gerador.py

Elemento	Recebe	Devolve	O que faz
sortear	quantidade e faixas	lista de Tarefa	Sorteia um conjunto de tarefas
comparar_em_lote	numero de cenarios e parametros	dicionario de medias	Compara os seis algoritmos sobre cenarios sorteados

`persistencia.py`

Elemento	Recebe	Devolve	O que faz
gravar	tarefas, caminho	—	Grava o conjunto em JSON
carregar	caminho	lista de Tarefa	Le e valida um conjunto gravado

6. Parametros configuraveis

Todos sao informados na janela do programa. Nenhum esta fixado no codigo-fonte.

Parametro	Faixa valida	Valor padrao	Observacao
Numero de tarefas	1 a 20	5	Define quantas linhas a tabela exhibe
Quantum <code>tq</code>	inteiro maior que <code>ttc</code>	2	So tem efeito sob Round-Robin
Custo da troca <code>ttc</code>	inteiro nao negativo	0	Vale para todos os algoritmos
Protocolo	nenhum, heranca, teto	nenhum	Heranca e teto sao alternativos
Passo do envelhecimento <code>alfa</code>	inteiro nao negativo	0	Zero desativa o envelhecimento

A combinacao de quantum menor ou igual ao custo da troca e recusada com mensagem, porque nesse caso a fatia inteira seria consumida pela troca e nenhum trabalho util seria realizado.

A eficiencia $E = tq / (tq + ttc)$ e propriedade da configuracao do escalonador, nao do conjunto de tarefas. Nos cinco algoritmos sem quantum ela nao esta definida, e o programa indica isso em vez de exibir um valor.

7. Funcionamento interno

O laco principal repete o ciclo abaixo enquanto houver tarefa nao concluida.

1. Montagem do conjunto de prontas. O motor reúne as tarefas que ja ingressaram, ainda nao concluíram e nao estao suspensas a espera do recurso.

2. Relógio ocioso. Se o conjunto estiver vazio e ainda houver tarefas por ingressar, o relógio salta diretamente para o proximo instante de ingresso, e o intervalo e registrado como ocioso no diagrama de tempo.

3. Recalculo das prioridades efetivas. Para cada tarefa pronta, a prioridade efetiva parte da prioridade base. Havendo envelhecimento, soma-se `alfa` multiplicado pelo tempo decorrido desde o ultimo despacho da tarefa, ou desde o seu ingresso caso ela ainda nao tenha executado. Em seguida, se um protocolo estiver habilitado e houver um detentor do recurso:

- sob **heranca**, e somente se existirem tarefas suspensas, o detentor assume a maior prioridade entre a sua propria e a das tarefas suspensas;
- sob **teto**, o detentor assume o valor do teto do recurso, independentemente de existir conflito.

A diferenca entre os dois protocolos e exatamente essa condicao: a heranca exige tarefas bloqueadas, o teto exige apenas um detentor.

4. Escolha. O conjunto de prontas e entregue a politica, que devolve a tarefa escolhida.

5. Troca de contexto. Se a tarefa escolhida difere da ultima que ocupou o processador, ocorre uma troca, inclusive no primeiro despacho. O relógio avanca `ttc` unidades sem trabalho util, e o intervalo e registrado como troca.

6. Orcamento da fatia. Sob Round-Robin, a fatia util vale `tq - ttc` quando houve troca, e `tq` quando a mesma

tarefa permanece no processador. Nos demais algoritmos não há orçamento.

7. Execução. Antes de cada unidade, o motor verifica se a tarefa atinge o início da sua seção crítica. Em caso afirmativo, se o recurso estiver livre ela o obtém; se estiver ocupado, ela é marcada como suspensa, sai do conjunto de prontas e da fila circular, e perde o processador. Caso contrário a tarefa executa uma unidade, o relógio avança e é executado e incrementado. Após a unidade, o motor verifica se a tarefa atingiu o fim da seção crítica, caso em que o recurso é devolvido e todas as tarefas suspensas voltam ao conjunto de prontas; e se a tarefa atingiu o seu `tp`, caso em que a conclusão é registrada.

8. Fim da fatia. A tarefa perde o processador ao concluir, ao ficar suspensa, ao esgotar o quantum, ou, nos algoritmos preemptivos sem quantum, ao deixar de ser a escolha da política após qualquer unidade de tempo. Ao esgotar o quantum, a tarefa volta à cauda da fila circular depois das que ingressaram naquele mesmo instante.

8. Convenções de simulação

São as decisões de modelagem que determinam os números produzidos. Sem elas, o mesmo conjunto de tarefas gera resultados diferentes.

#	Convenção
C1	O tempo é discreto e avança de uma em uma unidade. Nos cenários de referência a unidade é o segundo.
C2	Valor maior de prioridade significa prioridade mais alta.
C3	O desempate entre tarefas equivalentes se dá pelo menor instante de ingresso e, persistindo o empate, pelo menor identificador.
C4	Ocorre troca de contexto sempre que a tarefa despachada difere da última que ocupou o processador, inclusive no primeiro despacho.
C5	O custo da troca é descontado da fatia concedida, nunca somado a ela. Sob Round-Robin, a tarefa executa <code>tq - ttc</code> unidades úteis quando houve troca.
C6	A tarefa que esgota o quantum volta à cauda da fila depois das que ingressaram naquele mesmo instante.
C7	A seção crítica é medida no tempo de execução própria da tarefa. Início 1 e duração 4 significa que a tarefa obtém o recurso após executar 1 unidade útil e o mantém até ter executado 5.
C8	<code>tw = tt - tp</code> . O tempo de espera engloba fila, suspensão no recurso e troca de contexto.
C9	O teto de um recurso é calculado sobre as tarefas que declaram seção crítica nele, ainda que a disputa não chegue a ocorrer. Herança e teto são protocolos alternativos e não se combinam.
C10	O envelhecimento conta o tempo decorrido desde o último despacho da tarefa, ou desde o seu ingresso caso ela ainda não tenha executado. A prioridade volta ao valor base assim que a tarefa recebe o processador.

Dois consequências merecem registro explícito, porque são as que mais alteram os números:

- o custo da troca **sai de dentro** da fatia. Com `tq = 4` e `ttc = 1`, cada fatia ocupa 4 unidades de relógio e entrega 3 unidades de trabalho útil;
- a seção crítica é contada em **tempo de execução própria**, e não no relógio. Uma tarefa preemptada no meio da seção crítica continua detendo o recurso.

9. Dependências e ambiente

O simulador utiliza apenas a biblioteca padrão do Python. A interface gráfica usa `tkinter`, que acompanha a instalação padrão. Nenhuma biblioteca externa é necessária.

- Versão do Python: 3.10 ou superior;
- Bibliotecas externas: nenhuma;
- Geracao do executavel: `pyinstaller --onefile --windowed --name Simulador main.py`, executado na raiz do projeto. O arquivo resultante é copiado de `dist/` para a raiz do repositório.

O arquivo `validar.py` reproduz todos os cenários de referencia do enunciado e compara cada valor obtido com o esperado, o que permite confirmar que qualquer alteracao no codigo preserva os resultados.